Chapter 07: KNN

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [1]:
```
import matplotlib.pyplot as plt
import mlba
import pandas as pd
from sklearn import preprocessing
from sklearn.compose import ColumnTransformer # apply different preprocessir
from sklearn.model_selection import train_test_split, cross_val_score, Leave
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
%matplotlib inline
```

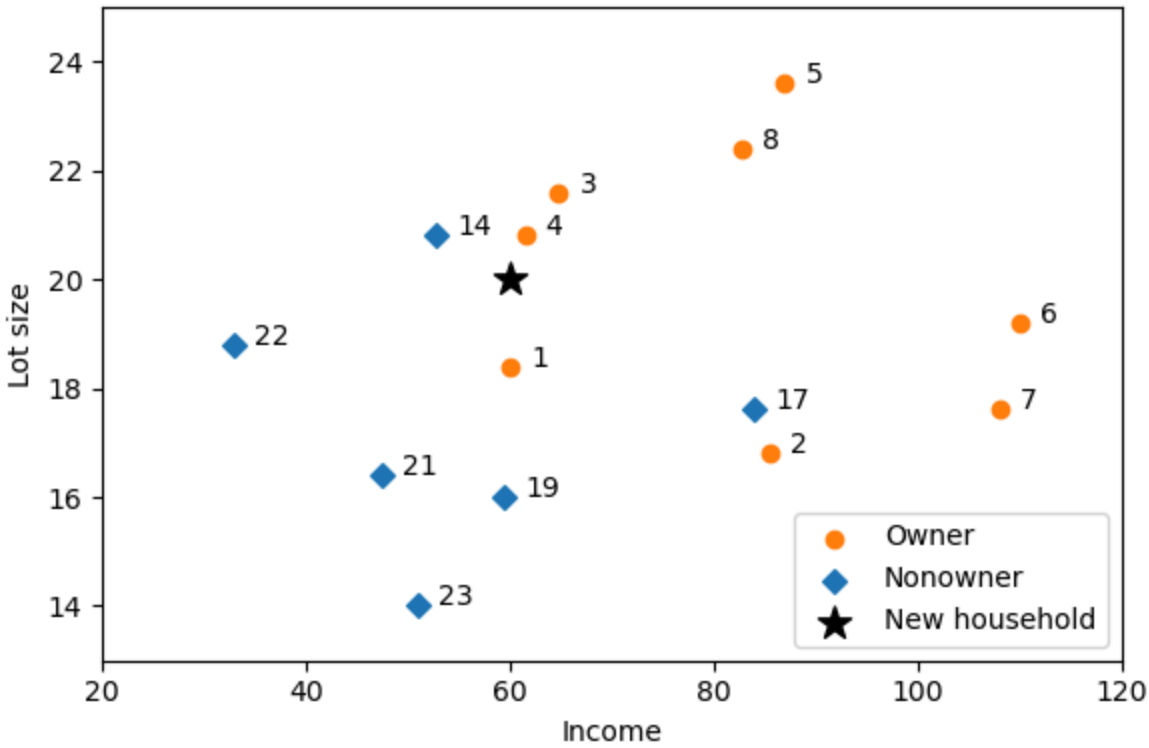
```
no display found. Using non-interactive Agg backend
```

Example: Riding Mowers

A riding-mower manufacturer would like to find a way of classifying families in a city into those likely to purchase a riding mower and those not likely to buy one, based on income and lot size. A pilot random sample is undertaken of 12 owners and 12 nonowners in the city. The data are shown in Table 7.1. We first partition the data into training data (14 households) and holdout data (10 households). Obviously, this dataset is too small for partitioning, which can result in unstable results, but we will continue with this partitioning for illustration purposes. A scatter plot of the training data is shown in Figure 7.1.

Figure 7.1 Loading and Partitioning the Riding Mower Data and Plotting Scatter Plot

In [2]:
```
#VB67
mower_df = mlba.load_data('RidingMowers.csv')
# add a column to label each observation for plot annotation
mower_df['Number'] = mower_df.index + 1

trainData, holdoutData = train_test_split(mower_df, test_size=0.4, random_st

# new household with specified income and lot size
newHousehold = pd.DataFrame([{'Income': 60, 'Lot_Size': 20}])

# Define a function to plot the dataset on a given object
def plotDataset(ax, data):
    # Plot owners as orange circles
    subset = data.loc[data['Ownership']=='Owner']
    ax.scatter(subset.Income, subset.Lot_Size, marker='o',
        label='Owner', color='C1')

    # Plot nonowners as blue diamonds
    subset = data.loc[data['Ownership']=='Nonowner']
    ax.scatter(subset.Income, subset.Lot_Size, marker='D',
        label='Nonowner', color='C0')

    # Annotate each point with its unique number
    for _, row in data.iterrows():
        ax.annotate(row.Number, (row.Income + 2, row.Lot_Size))

# Create a matplotlib figure
fig, ax = plt.subplots(figsize=(6, 4))

# Plot the training data
plotDataset(ax, trainData)

# Add the new household
ax.scatter(newHousehold.Income, newHousehold.Lot_Size, marker='*',
    label='New household', color='black', s=150)

ax.set_xlabel('Income'); ax.set_ylabel('Lot size')
ax.set_xlim(20, 120)
ax.set_ylim(13, 25)
ax.legend(loc=4)
plt.tight_layout()
plt.show()
```



In Table 7.2, we define the nearest neighbor model as a pipeline that first scales the data and then applies the KNN classifier. A pipeline combines data preprocessing with the model and ensures that all steps are applied using the same training data. Such a pipeline also makes it easier to apply the model to new data. Table 7.2 also shows how to determine the nearest neighbors. To do this, we first apply the preprocessing step (scaler) to transform the data into values that the model expects. The transformed data can then be used with the model method "kneighbors" to find the nearest neighbors and the distances to them.

Table 7.2 Running KNN

In [3]:
```
#VB67
# Defining predictor and outcome variables
predictors = ['Income', 'Lot_Size']
outcome = 'Ownership'

# combine scaling and k-NN classifier into model using a pipeline
model = Pipeline([
    ('scaler', preprocessing.StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=3)),
])
# Fit the pipeline on the training data
model.fit(trainData[predictors], trainData[outcome])
# Predict ownership for new household data
model.predict(newHousehold)
```

Out[3]:
```
array(['Owner'], dtype=object)
```

Determine the nearest neighbors of the new household

In [5]:
```
#VB67
zNewHousehold = model['scaler'].transform(newHousehold) # Use the standard s
distances, indices = model['knn'].kneighbors(zNewHousehold) # get the row ir
trainData.iloc[indices[0], :]
```

Out[5]:

	Income	Lot_Size	Ownership	Number
3	61.5	20.8	Owner	4
13	52.8	20.8	Nonowner	14
0	60.0	18.4	Owner	1

Choosing k

Table 7.3 Accuracy of KNN Predictions in Leave-One-Out Cross-Validation for Various Choices of k

In [6]:
```
#VB67
# Defining predictor and outcome variables
X = trainData[predictors]
y = trainData[outcome]
# Initializing list to store results
results = []
# Loop over different k values (1 to 13) for k-NN
for k in range(1, 14):
    # set the number of neighbors in the knn model
    model.set_params(knn__n_neighbors=k)
    accuracy = cross_val_score(model, X, y, cv=LeaveOneOut(), scoring='accur
    results.append({'k': k, 'accuracy': accuracy})
# Displaying results as a rounded DataFrame
pd.DataFrame(results).round(2)
```

Out[6]:

	k	accuracy
0	1	0.71
1	2	0.50
2	3	0.71
3	4	0.71
4	5	0.79
5	6	0.71
6	7	0.79
7	8	0.71
8	9	0.71
9	10	0.57
10	11	0.50
11	12	0.43
12	13	0.57

Table 7.4 Classifying a New Household Using the Best K = 5

In [7]:

```
#VB67
# Retrain with full dataset
knn = Pipeline([
    ('scaler', preprocessing.StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5)),
])
knn.fit(mower_df[predictors], mower_df[outcome])

# Standardize new household data
zNewHousehold = knn['scaler'].transform(newHousehold)
# Find distances and indices of 5 nearest neighbors
distances, indices = knn['knn'].kneighbors(zNewHousehold)

# Display predicted class and probabilities for the new household
print('Predicted class', knn.predict(newHousehold))
print('Probability', knn.predict_proba(newHousehold))

# Show distances and indices of neighbors
print('Distances',distances)
print('Indices', indices)

# show the five nearest neighbors used to classify the new housefold
print(mower_df.iloc[indices[0], :])
```

```
Predicted class ['Owner']
Probability [[0.2 0.8]]
Distances [[0.34532669 0.46448259 0.50133206 0.67307542 0.71721526]]
Indices [[ 3  8 13  0  2]]
    Income  Lot_Size Ownership  Number
3     61.5      20.8     Owner       4
8     69.0      20.0     Owner       9
13    52.8      20.8  Nonowner      14
0     60.0      18.4     Owner       1
2     64.8      21.6     Owner       3
```

Table 7.5 Column-Specific Transformations in Scikit-learn

Table 7.5 demonstrates how to define column-specific transformations using a ColumnTransformer. In this example, the columns Family and Car are converted using one-hot encoding, the column Income is standardized, and the column CreditCard is kept unchanged ('passthrough'). By default, OneHotEncoder will create a binary dummy for each category. Here, we used drop='if_binary' to drop one of the dummy variables if the column contains only two categories. This is the case for the Family column which is converted to a single column. Car, on the other hand, is converted to three columns.

In [8]:
```
#VB67
# Creating a sample DataFrame
df = pd.DataFrame({
    'Family': ['Single', 'Married', 'Married', 'Single', 'Married', 'Marriec
    'Car': ['SUV', 'SUV', 'Sports', 'Truck', 'Sports', 'Sports'],
    'Income': [49, 34, 11, 100, 45, 29],
    'CreditCard': [1, 1, 3, 1, 2, 2],
})
# Defining preprocessing steps using ColumnTransformer
preprocess = ColumnTransformer([
    ('encoder', preprocessing.OneHotEncoder(drop='if_binary'), ['Family', 'C
    ('standardize', preprocessing.StandardScaler(), ['Income']),
    ('unchanged', 'passthrough', ['CreditCard']),
]) # a preprocessing pipeline using ColumnTransformer when different columns
preprocess.fit_transform(df).round(4)
```

Out[8]:
```
array([[ 1.    ,  1.    ,  0.    ,  0.    ,  0.157 ,  1.    ],
       [ 0.    ,  1.    ,  0.    ,  0.    , -0.3864,  1.    ],
       [ 0.    ,  0.    ,  1.    ,  0.    , -1.2194,  3.    ],
       [ 1.    ,  0.    ,  0.    ,  1.    ,  2.0042,  1.    ],
       [ 0.    ,  0.    ,  1.    ,  0.    ,  0.0121,  2.    ],
       [ 0.    ,  0.    ,  1.    ,  0.    , -0.5675,  2.    ]])
```

In [ ]: